Linear Algebra and Calculus in Full

Extended mathematics notes Sources: listed under References

What this file is really about:

$$\text{numbers} \rightarrow \text{vectors and matrices} \rightarrow \text{rates of change} \rightarrow \text{everything a model computes}$$

How to use this file. This is a reference that starts from the beginning. Section A1 maps each advanced topic to the maths it needs. If a formula elsewhere in the course looks unfamiliar, find its name here. Part B checks the main identities numerically.

Step-up path

You need. Arithmetic and a little algebra.

1. Multiply matrices.
2. Find eigenvalues.
3. Apply each derivative rule.
4. Differentiate a quadratic form.
5. Evaluate an integral by parts.

You can stop when you can read any formula in the course.

Contents

Part A Maths From the Ground Up

A1 A map from topics to the maths they need

To understand	You need
Linear and ridge regression	Matrix products, transpose, inverse, gradient of a quadratic, projection
PCA and SVD	Eigenvalues, orthogonality, spectral theorem, variance, Lagrange multipliers
Logistic regression	Logarithm, chain rule, Hessian, convexity, Bernoulli likelihood
SVMs and kernels	Dot product, Lagrange duality and KKT, positive semidefinite matrices
Trees and boosting	Entropy, Taylor expansion to second order, weighted averages
Clustering and mixtures	Norms, Gaussian density, Jensen's inequality, conditional expectation
Gaussian processes	Multivariate normal, conditioning, Cholesky, kernels
Backpropagation	Chain rule, Jacobians, vector-Jacobian products
Optimisers	Gradients, Taylor expansion, eigenvalues of the Hessian, moving averages
Convolutions	Sums over indices, Fourier idea, tensor shapes
Attention	Dot product, softmax, matrix products, masks
VAEs	Expectation, KL divergence, Jensen, reparameterisation
GANs	Minimax, Jensen-Shannon and Wasserstein distances
Normalising flows	Change of variables, Jacobian determinant
Diffusion and score models	Gaussians, Markov chains, stochastic differential equations
Reinforcement learning	Expectation, Markov chains, fixed points, contraction
Graph networks	Adjacency matrix, Laplacian, eigenvectors
Scene rendering	Rotation matrices, homogeneous coordinates, numerical integration

The probability entries are in the companion note on probability and statistics. The stochastic, Fourier and optimisation theory entries are in the note on advanced maths.

A2 Notation

- $\sum_{i=1}^{n} a_i$ adds a_1 to a_n, and $\prod_{i=1}^{n} a_i$ multiplies them.
- $x \in A$ means x is in the set A. $A \cup B$ is union, $A \cap B$ intersection, and A^c the complement.
- $\mathbb{R}^d$ is the set of real vectors of length d.
- $f : \mathbb{R}^d \to \mathbb{R}$ says the function takes a d-vector and returns a number.
- $\arg\min_x f(x)$ is the input that minimises f. $\min_x f(x)$ is the smallest value itself.

A3 Exponentials and logarithms

The **exponential function** e^x is its own derivative. Its laws are $e^{a+b} = e^a e^b$ and $(e^a)^b = e^{ab}$. The **logarithm** undoes it: $\ln e^x = x$. Its laws are

$$\ln(ab) = \ln a + \ln b, \qquad \ln(a^b) = b \ln a, \qquad \log_b x = \frac{\ln x}{\ln b}.$$

Logs turn products into sums. That is why likelihoods are maximised on a log scale, where they also avoid underflow to zero.

Log-sum-exp. To add probabilities held as logs, $\ln \sum_i e^{a_i}$, factor out the largest term m: $m + \ln \sum_i e^{a_i - m}$. No term then overflows.

A4 Vectors

A vector is a list of numbers, and also an arrow. **Adding vectors** works entry by entry. Scaling multiplies each entry. The **dot product** is $x^\top y = \sum_i x_i y_i = \|x\| \|y\| \cos\theta$. If it is zero, the vectors are **orthogonal**.

Norms measure size. For $p \geq 1$ the p-norm is $\|x\|_p = (\sum_i |x_i|^p)^{1/p}$. So $\|x\|_1$ adds absolute values, $\|x\|_2$ is the usual length, and $\|x\|_\infty$ is the largest entry. Every norm obeys the **triangle inequality**, $\|x + y\| \leq \|x\| + \|y\|$.

The **Cauchy-Schwarz** inequality is $|x^\top y| \leq \|x\| \|y\|$. It is why **cosine similarity** $x^\top y / (\|x\| \|y\|)$ lies between -1 and 1.

A5 Span, independence and basis

A **linear combination** is $\sum_i c_i v_i$. The **span** of some vectors is all their linear combinations. Vectors are **linearly independent** if no vector is a combination of the others, which means $\sum_i c_i v_i = 0$ forces every $c_i = 0$. A **basis** is an independent set that spans the space. Every basis of a space has the same size, called its **dimension**.

A6 Matrices and matrix multiplication

An $m \times n$ matrix has m rows and n columns. A **linear map** is a function with $f(ax + by) = af(x) + bf(y)$, and every one is multiplication by a matrix. The product of A ($m \times n$) and B ($n \times p$) is

$$(AB)_{ij} = \sum_k A_{ik} B_{kj}.$$

The result is $m \times p$. Multiplying applies B first and then A. In general $AB \neq BA$, but $(AB)C = A(BC)$. The **identity matrix** I has ones on the diagonal, and $AI = IA = A$.

The **transpose** swaps rows and columns, and $(AB)^\top = B^\top A^\top$. The **trace** is the sum of the diagonal, and $\operatorname{Tr}(AB) = \operatorname{Tr}(BA)$. A **block matrix** multiplies like ordinary matrices when the block shapes agree.

An **einsum** string names the indices: `ij,jk->ik` is a matrix product and `ii->` is a trace. Repeated indices that vanish from the output are summed.

A7 Inverse, determinant and rank

A^{-1} undoes A, so $AA^{-1} = I$. For a 2×2 matrix:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}.$$

The **determinant** is the factor by which a matrix scales area or volume. If $\det A = 0$, some direction is flattened and A has no inverse. Also $\det(AB) = \det A \det B$, and for a triangular matrix it is the product of the diagonal.

The **rank** is the dimension of the **column space**, the span of the columns. The **null space** is every x with $Ax = 0$. They obey rank-nullity: $\operatorname{rank}(A) + \dim \operatorname{null}(A) = n$.

Woodbury's identity (the matrix inversion lemma) turns a large inverse into a small one when an update has low rank:

$$(A + UCV)^{-1} = A^{-1} - A^{-1}U(C^{-1} + VA^{-1}U)^{-1}VA^{-1}.$$

A8 Solving linear systems and factorisations

Never compute an inverse if you only need to solve $Ax = b$. Factor A instead.

- **LU**: $A = LU$ with lower and upper triangular factors. Two cheap triangular solves follow.
- **Cholesky**: if A is symmetric positive definite, $A = LL^\top$. It is about twice as fast, and it gives $\det A = \prod_i L_{ii}^2$. It is also how a multivariate normal is sampled.
- **QR**: $A = QR$ with orthonormal columns in Q. It solves least squares stably.

Gram-Schmidt builds orthonormal vectors from any basis. Take each vector, subtract its projections onto the earlier results, and normalise:

$$u_k = v_k - \sum_{j<k} (q_j^\top v_k) q_j, \qquad q_k = u_k / \|u_k\|.$$

The **pseudo-inverse** A^+ gives the minimum-norm least-squares solution $x = A^+ b$. It is computed from the SVD as $V\Sigma^+ U^\top$.

The **condition number** $\kappa = \sigma_{\max}/\sigma_{\min}$ says how much errors are magnified.

A9 Projection and change of basis

The projection of y onto the column space of X is

$$\hat{y} = X(X^\top X)^{-1} X^\top y.$$

The residual $y - \hat{y}$ is orthogonal to every column. If the columns of Q are orthonormal, the formula is simply $QQ^\top y$.

To write x in a new basis with columns B, solve $Bc = x$. An orthonormal basis gives $c = B^\top x$ directly. In the new basis, a map A becomes $B^{-1}AB$.

A **rotation matrix** in the plane is $R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$. It is orthogonal, $R^\top R = I$, and preserves lengths.

A10 Eigenvalues, the spectral theorem and diagonalisation

A nonzero v is an **eigenvector** if $Av = \lambda v$, and λ is its **eigenvalue**. They solve $\det(A - \lambda I) = 0$. Always $\operatorname{Tr}(A) = \sum_i \lambda_i$ and $\det A = \prod_i \lambda_i$.

If $A = V \Lambda V^{-1}$, it is **diagonalised**. Powers become easy: $A^t = V \Lambda^t V^{-1}$, which explains why gradient descent and Markov chains are governed by eigenvalues.

The **spectral theorem**: a real symmetric matrix has real eigenvalues and an orthonormal basis of eigenvectors, so $A = Q \Lambda Q^\top$.

A **quadratic form** is $x^\top A x$. A is **positive definite** if $x^\top A x > 0$ for all nonzero x, and **positive semidefinite** if ≥ 0. For symmetric A, those mean all eigenvalues are positive or non-negative. Covariance matrices and kernel matrices are positive semidefinite.

Power iteration finds the largest eigenvector: repeat $x \leftarrow Ax / \|Ax\|$. Convergence depends on $|\lambda_2 / \lambda_1|$.

A11 Singular value decomposition and matrix norms

Every matrix can be written $A = U \Sigma V^\top$, with orthonormal U, V and non-negative singular values on the diagonal of Σ. The right singular vectors are eigenvectors of $A^\top A$, and $\sigma_i^2 = \lambda_i(A^\top A)$. Truncating to the top k gives the best rank-k approximation.

The **spectral norm** $\|A\|_2 = \sigma_{\max}$ is the most a unit vector can be stretched. The **Frobenius norm** $\|A\|_F = \sqrt{\sum_{ij} A_{ij}^2} = \sqrt{\sum_i \sigma_i^2}$.

The **Kronecker product** $A \otimes B$ replaces each entry A_{ij} by the block $A_{ij} B$. Its eigenvalues are all the products $\lambda_i(A) \lambda_j(B)$.

The **matrix exponential** is $e^A = \sum_k A^k / k!$, the solution operator of $\dot{x} = Ax$.

A12 Limits and continuity

$\lim_{x \to a} f(x) = L$ means $f(x)$ can be made as close to L as you like by taking x close to a. f is **continuous** at a if $\lim_{x \to a} f(x) = f(a)$. Two limits appear constantly: $\lim_{h \to 0} \frac{e^h - 1}{h} = 1$ and $\lim_{n \to \infty} (1 + x/n)^n = e^x$.

L'Hôpital's rule: if a ratio is $0/0$, then under mild conditions $\lim f/g = \lim f'/g'$. It gives $\lim_{p \to 0} p \ln p = 0$, which is why entropy treats $0 \ln 0$ as 0.

A13 Derivatives and their rules

$$f'(x) = \lim_{h \to 0} \frac{f(x + h) - f(x)}{h}.$$

Rule	Formula
Power rule	$(x^n)' = n x^{n-1}$
Constant multiple and sum	$(af + g)' = af' + g'$
Product rule	$(fg)' = f'g + fg'$
Quotient rule	$(f/g)' = (f'g - fg')/g^2$
Chain rule	$(f \circ g)'(x) = f'(g(x))\, g'(x)$
Exponential	$(e^x)' = e^x$
Logarithm	$(\ln x)' = 1/x$
Sigmoid	$\sigma' = \sigma(1 - \sigma)$

The **mean value theorem** says that for smooth f, some c in (a, b) has $f'(c) = \frac{f(b) - f(a)}{b - a}$. It underlies Lipschitz arguments.

A14 Taylor expansion

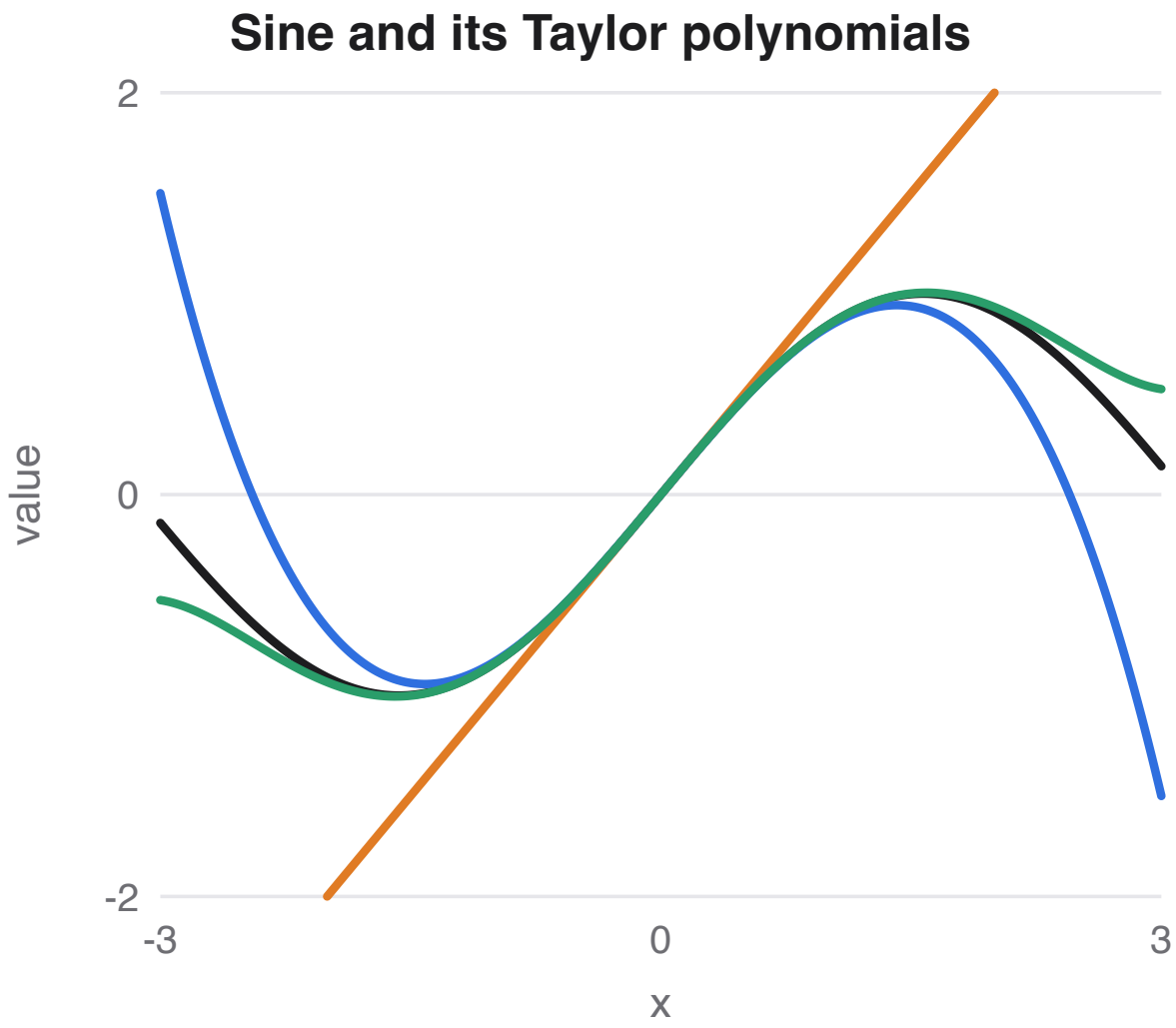
Near a point a,

$$f(x) = f(a) + f'(a)(x-a) + \frac{f''(a)}{2}(x-a)^2 + \cdots$$

In many variables, to second order:

$$f(x+\delta) \approx f(x) + \nabla f^\top \delta + \tfrac{1}{2}\delta^\top H \delta.$$

Setting the gradient of that quadratic to zero gives Newton's step, $\delta = -H^{-1}\nabla f$. XGBoost's gain formula comes from the same expansion.



A15 Partial derivatives, gradients, Jacobians and Hessians

A **partial derivative** changes one input and holds the rest. The **gradient** collects them, $\nabla f = (\partial_1 f, \ldots, \partial_d f)$. The **directional derivative** along unit u is $\nabla f^\top u$, which is largest in the direction of the gradient. So the gradient is the steepest ascent, and it is perpendicular to level sets.

For $f : \mathbb{R}^n \to \mathbb{R}^m$ the **Jacobian** is the $m \times n$ matrix $J_{ij} = \partial f_i / \partial x_j$. The chain rule becomes the matrix product $J_{f \circ g} = J_f J_g$. The **Hessian** $H_{ij} = \partial_i \partial_j f$ holds second derivatives. It is symmetric for smooth functions.

A16 Matrix calculus

$f(x)$	$\nabla_x f$
$a^\top x$	a
$x^\top A x$	$(A + A^\top)x$, which is $2Ax$ when A is symmetric
$\|Ax - b\|^2$	$2A^\top(Ax - b)$
$\ln \det X$	$X^{-\top}$ with respect to matrix X
$\mathrm{Tr}(AX)$	$A^\top$ with respect to matrix X

Setting the third row to zero gives the normal equation, $A^\top A x = A^\top b$.

A17 Integrals

An **integral** $\int_a^b f(x)\,dx$ is the area under f as the limit of thin rectangles. The **fundamental theorem of calculus** connects it to derivatives: if $F' = f$, then $\int_a^b f = F(b) - F(a)$.

Substitution reverses the chain rule. With $u = g(x)$, $\int f(g(x))g'(x)\,dx = \int f(u)\,du$.

Integration by parts reverses the product rule:

$$\int u\,dv = uv - \int v\,du.$$

It proves $\mathbb{E}[X]$ formulas and Stein's identity, and it is the step behind score matching, which removes the unknown score.

A **double integral** integrates over a region. For rectangles and non-negative f, Fubini's theorem allows either order.

A18 Special integrals and functions

The **Gaussian integral** is $\int_{-\infty}^{\infty} e^{-x^2/2}\,dx = \sqrt{2\pi}$, which sets the normal density's constant.

The **gamma function** $\Gamma(z) = \int_0^\infty t^{z-1}e^{-t}\,dt$ extends the factorial: $\Gamma(n) = (n-1)!$ and $\Gamma(z+1) = z\Gamma(z)$. The **beta function** $B(a,b) = \Gamma(a)\Gamma(b)/\Gamma(a+b)$ normalises the beta distribution.

A19 Change of variables

For an invertible $y = g(x)$, densities pick up the Jacobian determinant:

$$p_Y(y) = p_X(g^{-1}(y))\left|\det \frac{\partial g^{-1}}{\partial y}\right|.$$

The determinant accounts for how g stretches volume. Normalising flows are built entirely on this formula.

Leibniz's rule moves a derivative inside an integral when limits are fixed, $\frac{d}{d\theta}\int f(x,\theta)\,dx = \int \partial_\theta f(x,\theta)\,dx$. It justifies the gradient of an expected loss.

The **implicit function theorem** gives derivatives of a quantity defined by an equation. If $F(z,\theta) = 0$, then $\frac{dz}{d\theta} = -(\partial_z F)^{-1}\partial_\theta F$.

A20 Lagrange multipliers and KKT

To optimise $f(x)$ subject to $g(x) = 0$, form $L = f + \lambda g$ and set its gradient to zero. At the solution the gradients of f and g are parallel. For maximum-variance PCA this gives $Sv = \lambda v$.

With inequalities $g_i(x) \le 0$, the **KKT conditions** are:

- stationarity: $\nabla f + \sum_i \lambda_i \nabla g_i = 0$;
- primal feasibility: $g_i \le 0$;
- dual feasibility: $\lambda_i \ge 0$;
- complementary slackness: $\lambda_i g_i = 0$.

Complementary slackness is why only support vectors have nonzero α_i.

A21 Convexity and Jensen's inequality

A set is convex if the segment between any two of its points stays inside it. f is convex if $f(tx + (1-t)y) \leq tf(x) + (1-t)f(y)$. A twice-differentiable f is convex exactly when its Hessian is positive semidefinite. Every local minimum of a convex function is global.

Jensen's inequality: for convex f, $f(\mathbb{E}[X]) \leq \mathbb{E}[f(X)]$. It proves the ELBO, that KL is non-negative, and that EM never lowers the likelihood.

A22 Proof tools and complexity

Induction proves a base case, then that each case implies the next. **Proof by contradiction** assumes the opposite and derives something impossible.

Big-O describes growth, ignoring constants: $f = O(g)$ if $f \leq cg$ for large n. A matrix product of $n \times n$ matrices costs $O(n^3)$, and attention over n tokens costs $O(n^2)$.

Part B In Code

B1 Matrix identities

```
import numpy as np
rng = np.random.default_rng(0)
A, B = rng.normal(size=(3, 3)), rng.normal(size=(3, 3))
assert np.allclose((A @ B).T, B.T @ A.T)
assert np.isclose(np.trace(A @ B), np.trace(B @ A))
assert np.isclose(np.linalg.det(A @ B), np.linalg.det(A) * np.linalg.det(B))
w = np.linalg.eigvals(A)
assert np.isclose(w.sum().real, np.trace(A)) and np.isclose(w.prod().real, np.linalg.det(A)
assert np.allclose(np.einsum('ij,jk->ik', A, B), A @ B)
# Woodbury
M = np.eye(5) * 2.0; U = rng.normal(size=(5, 2)); C = np.eye(2)
lhs = np.linalg.inv(M + U @ C @ U.T)
iM = np.linalg.inv(M)
rhs = iM - iM @ U @ np.linalg.inv(np.linalg.inv(C) + U.T @ iM @ U) @ U.T @ iM
assert np.allclose(lhs, rhs); print('identities hold')
```

B2 Factorisations and Gram-Schmidt

```
import numpy as np
rng = np.random.default_rng(1)
X = rng.normal(size=(8, 3))
def gram_schmidt(V):
    Q = np.zeros_like(V)
    for k in range(V.shape[1]):
        u = V[:, k] - Q[:, :k] @ (Q[:, :k].T @ V[:, k]); Q[:, k] = u / np.linalg.norm(u)
    return Q
Q = gram_schmidt(X); assert np.allclose(Q.T @ Q, np.eye(3))
S = X.T @ X; L = np.linalg.cholesky(S); assert np.allclose(L @ L.T, S)
assert np.isclose(np.prod(np.diag(L))**2, np.linalg.det(S))
y = rng.normal(size=8)
b = np.linalg.pinv(X) @ y; r = y - X @ b
assert np.allclose(X.T @ r, 0, atol=1e-10); print('factorisations and projection hold')
```

B3 Power iteration against the eigendecomposition

```
import numpy as np
rng = np.random.default_rng(2)
A = rng.normal(size=(6, 6)); A = A @ A.T
x = rng.normal(size=6)
for _ in range(4000): x = A @ x; x /= np.linalg.norm(x)
print(round(x @ A @ x, 6), round(np.linalg.eigvalsh(A)[-1], 6))
assert np.isclose(x @ A @ x, np.linalg.eigvalsh(A)[-1])
U, s, Vt = np.linalg.svd(A)
assert np.isclose(np.linalg.norm(A, 2), s[0]) and np.isclose(np.linalg.norm(A), np.sqrt((s*
```

B4 Gradients by finite differences

```
import numpy as np
rng = np.random.default_rng(3)
A = rng.normal(size=(4, 4)); b = rng.normal(size=4); x = rng.normal(size=4)
def num(f, x, e=1e-6):
    return np.array([(f(x + e * d) - f(x - e * d)) / (2 * e) for d in np.eye(len(x))])
assert np.allclose(num(lambda v: v @ A @ v, x), (A + A.T) @ x, atol=1e-5)
assert np.allclose(num(lambda v: np.sum((A @ v - b)**2), x), 2 * A.T @ (A @ x - b), atol=1e
# Taylor: the second-order error shrinks like the cube of the step.
f = lambda t: np.exp(np.sin(t)); t = 0.7
d1 = np.cos(t) * f(t); d2 = (np.cos(t)**2 - np.sin(t)) * f(t)
errs = [abs(f(t + h) - (f(t) + d1 * h + d2 * h * h / 2)) for h in (0.1, 0.05)]
print(round(errs[0] / errs[1], 2)); assert 6.5 < errs[0] / errs[1] < 9.5
```

B5 Integrals, numerically

```
import numpy as np
from scipy.special import gamma, beta
x = np.linspace(-12, 12, 400001); dx = x[1] - x[0]
assert np.isclose(np.sum(np.exp(-x * x / 2)) * dx, np.sqrt(2 * np.pi))
t = np.linspace(1e-9, 60, 2000001); dt = t[1] - t[0]
assert np.isclose(np.sum(t**4 * np.exp(-t)) * dt, gamma(5), atol=1e-3) and gamma(5) == 24
# Integration by parts on the interval [0, 1], u = x and dv = e^x dx.
g = np.linspace(0, 1, 2000001); h = g[1] - g[0]
lhs = np.trapezoid(g * np.exp(g), g); rhs = 1 * np.e - np.trapezoid(np.exp(g), g)
assert np.isclose(lhs, rhs); assert np.isclose(beta(2, 3), gamma(2) * gamma(3) / gamma(5))
print('integrals agree')
```

B6 Change of variables

```
import numpy as np
rng = np.random.default_rng(5)
x = rng.normal(size=4_000_000); y = np.exp(x)       # y = g(x) is log-normal
a, b = 1.0, 1.2
empirical = np.mean((y > a) & (y < b)) / (b - a)
mid = np.linspace(a, b, 2001)
formula = np.trapezoid(np.exp(-np.log(mid)**2 / 2) / np.sqrt(2 * np.pi) / mid, mid) / (b -
print(round(empirical, 4), round(formula, 4)); assert abs(empirical - formula) < 0.003
```

Part C Practice

P1. What is the shape of AB if A is 3×2 and B is 2×5?

3×5.

P2. Find the eigenvalues of $\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$.

$\det(A - \lambda I) = (2-\lambda)^2 - 1 = 0$, so $\lambda = 1, 3$.

P3. Differentiate $x^2 e^{3x}$.

By the product and chain rules, $2xe^{3x} + 3x^2 e^{3x}$.

P4. Compute $\int_0^1 x e^x \, dx$.

By parts, $[xe^x]_0^1 - \int_0^1 e^x \, dx = e - (e-1) = 1$.

P5. What is $\Gamma(6)$?

$5! = 120$.

P6. If $Y = 2X$ and X has density p, what is $p_Y(y)$?

$p(y/2) \cdot \frac{1}{2}$.

P7. Minimise $x^2 + y^2$ subject to $x + y = 2$.

$L = x^2 + y^2 + \lambda(x + y - 2)$ gives $x = y = 1$, with value 2.